



Nombre: Luis Alberto Vargas González

Fecha: 04/07/2025

Actividad: U7 A1 Problema de la mochila, parentización óptima, corte de cuerda con programación dinámica.

Clase: Análisis y Diseño de Algoritmos.

Maestría en Ciberseguridad.

INTRODUCCIÓN.

En este programa se realizaron 3 funciones en donde cada una de ellas resuelve un algoritmo distinto y un programa distinto , cabe recalcar que dichas funciones no se llaman entre sí, como en otros programas pues cada una resuelve un problema dado en la actividad.

Es importante también señalar que casi al final del programa se declaran los datos a resolver de cada problema, que son los mismos datos que nos fueron proporcionados por el documento de la tarea.

Al final del programa simplemente lo que se hace es mostrar los resultados de cada problema computado; el de la mochila, el de corte de cuerda, y el de parentización óptima.

Es importante señalar que fue bastante desafiante hacer este programa ya que se tuvo que probar varias veces cada versión de cada función , por lo que la correcta computación de los problemas tomó aun más tiempo del que me imaginé.

Código.

```
# PROBLEMA 1: MOCHILA (0/1 KNAPSACK)
def knapsack(values, weights, capacity):
    n = len(values)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w],
                               dp[i - 1][w -
weights[i - 1]] + values[i - 1])
            else:
                dp[i][w] = dp[i - 1][w]

    # Recuperar objetos seleccionados
    selected = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i)
            w -= weights[i - 1]

    selected.reverse()
    return dp[n][capacity], selected

# PROBLEMA 2: CORTE DE CUERDA (CUTTING ROD)
def cut_rod(prices, length):
    dp = [0] * (length + 1)
    cuts = [0] * (length + 1)

    for i in range(1, length + 1):
```

```

        max_val = float('-inf')
        for j in range(1, i + 1):
            if j <= len(prices):
                if prices[j - 1] + dp[i - j] >
max_val:
                    max_val = prices[j - 1] + dp[i
- j]
                    cuts[i] = j
        dp[i] = max_val

# Reconstruir cortes
decomposition = []
while length > 0:
    decomposition.append(cuts[length])
    length -= cuts[length]

return dp[-1], decomposition

```

PROBLEMA 3: PARENTIZACIÓN ÓPTIMA (MATRIX CHAIN ORDER)

```
import sys
```

```

def matrix_chain_order(p):
    n = len(p) - 1
    m = [[0] * n for _ in range(n)]
    s = [[0] * n for _ in range(n)]

    for l in range(2, n + 1):
        for i in range(n - l + 1):
            j = i + l - 1
            m[i][j] = sys.maxsize
            for k in range(i, j):

```

```

        q = m[i][k] + m[k + 1][j] + p[i] *
p[k + 1] * p[j + 1]
        if q < m[i][j]:
            m[i][j] = q
            s[i][j] = k

def build_solution(s, i, j):
    if i == j:
        return f"A{i + 1}"
    else:
        return f"({build_solution(s, i,
s[i][j]))} x {build_solution(s, s[i][j] + 1, j)})"

return m[0][n - 1], build_solution(s, 0, n - 1)

# DATOS DEL DOCUMENTO
# Mochila
values = [79, 32, 47, 18, 26, 85, 33, 40, 45, 59]
weights = [85, 26, 48, 21, 22, 95, 43, 45, 55, 52]
capacity = 140

# Corte de cuerda
prices = [1, 4, 10, 12, 15, 20, 21, 32, 31, 41, 51]
length = 11

# Parentización
p = [5, 10, 3, 12, 5, 50, 6]

# RESULTADOS
valor_mochila, objetos_mochila = knapsack(values,
weights, capacity)
precio_cuerda, cortes_cuerda = cut_rod(prices,
length)

```

```
multiplicaciones, parentizacion =  
matrix_chain_order(p)  
  
# MOSTRAR RESULTADOS  
print("==== PROBLEMA DE LA MOCHILA ====")  
print("Valor óptimo:", valor_mochila)  
print("Objetos seleccionados:", objetos_mochila)  
  
print("\n==== PROBLEMA DE CORTE DE CUERDA ====")  
print("Precio máximo:", precio_cuerda)  
print("Cortes óptimos:", cortes_cuerda)  
  
print("\n==== PROBLEMA DE PARENTIZACIÓN ÓPTIMA  
====")  
print("Mínimo de multiplicaciones escalares:",  
multiplicaciones)  
print("Parentización óptima:", parentizacion)
```

Resultados.

```
PS C:\Users\luisv> & C:/Users/luisv/AppData/Local/Programs/Python/Python312/python.exe c:/Users/luisv/Desktop/DP.py
==== PROBLEMA DE LA MOCHILA ====
Valor óptimo: 143
Objetos seleccionados: [4, 5, 8, 10]

==== PROBLEMA DE CORTE DE CUERDA ====
Precio máximo: 51
Cortes óptimos: [11]

==== PROBLEMA DE PARENTIZACIÓN ÓPTIMA ====
Mínimo de multiplicaciones escalares: 2010
Parentización óptima: ((A1 x A2) x ((A3 x A4) x (A5 x A6)))
PS C:\Users\luisv>
```

Explicación de resultados:

1. En el algoritmo de la mochila obtuvimos que la solución óptima de agregar artículos es : tomar el articulo 4,5,8 y 10 , esto debido a que ellos sumados nos dan 140 , que es el limite de peso que podemos agregar a la mochila, ya que si bien el objeto 4 y 5 contienen los valores más bajos, contienen también los pesos y tamaños más bajos por igual, lo cual nos permite agregar el artículo 8 y 10 para poder ajustarnos al peso máximo de la mochila que es 140. Esto es asi debido a que el algoritmo busca que no se exceda estrictamente el peso de 140 y maximizar la suma de los valores con la mayor cantidad de artículos tomados.
2. En el algoritmo de corte de cuerda se llegó al resultado más simple y quizás el más obvio también , la solución mas optima para poder vender la cuerda de tamaño 11 es simplemente venderla sin cortarla, ya que si la cortamos en pedazos de “metros” que sumados den 11 (10+1, 9+2, 7+4) la suma de los precios de dichos metros no iguala a el precio de vender la cuerda sin cortar.
3. Para el resultado de la parentización , obtuvimos que la forma más óptima de parentizar la multiplicación de matrices y minimizar el número de multiplicaciones escalares es de la forma : **$((A1 \times A2) \times ((A3 \times A4) \times (A5 \times A6)))$**

**(A5 x A6))), y nos dan 2010 operaciones escalares, si por el contrario
hubiésemos elegido una forma como esta: (((A1 x A2) x A3) x A4) x
(A5 x A6) nos daría 2280 operaciones**

Conclusión:

Como pudimos observar la realización de estos algoritmos conllevó una cierta complejidad pues en sí los problemas de optimización son complejos la primera vez que se abordan , la cual fue mi caso, pero sin embargo , el poder ir probando todas las posibilidades de las funciones me dio la oportunidad de aprender de manera más profunda como es que funcionan internamente estos algoritmos y poder programarlos de manera más precisa, y basándonos en los pseudocódigos proporcionados , pues un poco más sencillo se vuelve el programarlos.